团队辩论记录

讨论时间

2026 年 4 月 30 日

参与成员

- 成员 A
- 成员 B
- 成员 C
- 成员 D

讨论问题 1：是否采用微服务架构？

支持微服务的观点

- 用户、商品、订单、消息等模块边界看起来比较清晰。
- 微服务架构具有更好的可扩展性。
- 如果未来用户量增加，可以单独扩展商品服务或订单服务。
- 架构文档上看起来更“高级”。

反对微服务的观点

- 当前项目是课程项目，主要目标是在 10 周内完成可运行系统，而不是支撑真实大规模并发。
- 团队没有完整微服务开发经验，学习成本较高。
- 微服务会引入服务间通信、接口联调、部署、服务注册、网关、分布式事务等额外复杂度。
- 项目规模较小，过早拆分服务可能导致开发效率下降。
- 微服务的收益在当前阶段不明显，反而可能增加失败风险。

讨论结论

团队认为，微服务架构虽然具有较强扩展性，但对本项目而言属于“过度设计”。当前不采用微服务作为主体架构。

讨论问题 2：是否采用事件驱动架构？

支持事件驱动的观点

- 订单创建、消息通知、评价提醒等功能适合使用事件触发。
- 模块之间可以降低直接依赖。
- 后续如果增加推荐、通知、日志分析等功能，事件机制会比较方便。

反对事件驱动的观点

- 事件驱动需要额外设计事件类型、事件发布与消费机制。
- 如果引入消息队列，会增加部署和调试成本。
- 本项目大部分功能是 CRUD 和同步请求，事件驱动不是主要矛盾。
- 异步流程会增加测试难度，不利于课程项目按时交付。

讨论结论

团队认为，事件驱动思想可以局部借鉴，例如在订单创建后进行消息通知，但不作为系统主体架构。当前阶段可以先通过同步调用或简单的内部事件机制实现，暂不引入复杂消息队列。

讨论问题 3：是否采用分层架构 + MVC？

支持观点

- 结构清晰，适合用户、商品、订单、评价等典型 Web 系统。
- 开发复杂度低，便于团队成员快速上手。
- Controller、Service、Repository 分层有利于维护。
- 前后端分离后，前端和后端可以并行开发。
- 后端内部可以按照业务模块划分，避免代码混乱。
- 10 周内完成的可行性最高。

反对观点

- 单体架构扩展能力不如微服务。
- 如果缺乏模块边界约束，后期代码可能变得臃肿。
- 所有功能部署在一个应用中，局部修改可能影响整体系统。

解决措施

- 在单体内部采用清晰的模块化设计。
- 严格遵守 Controller、Service、Repository 分层。
- 不允许跨模块直接访问数据库表，尽量通过 Service 接口调用。
- 保持用户、商品、订单、消息、评价等模块边界清晰。
- 在架构文档中说明未来可演进为微服务。

讨论结论

团队一致认为，分层架构 + MVC 模式的模块化单体应用最适合本项目。

最终架构选择

最终选择

本项目最终选择：

> **前后端分离 + 分层架构 + MVC 模式的模块化单体架构**

其中：

- 前端负责页面展示、用户交互和接口调用；
- 后端采用 Controller、Service、Repository/Mapper 分层；
- 后端内部按业务划分为用户模块、商品模块、订单模块、评价模块、消息模块、收藏模块等；
- 数据统一存储在关系型数据库中；
- 后续如有需要，可逐步将高耦合度较低的模块拆分为独立服务。

最终选择理由

1. 与项目规模匹配

校园二手交易平台功能较完整，但并未达到必须使用微服务的复杂程度。采用模块化单体能够满足当前业务需求。

2. 10 周内可落地性最高

课程项目时间有限，团队需要优先完成可运行系统。分层单体架构可以减少部署、调试和接口联调成本。

3. 团队学习成本较低

团队成员已有基础前后端开发经验，MVC 和分层架构更容易理解和实践。

4. 便于模块划分和文档编写

用户模块、商品模块、订单模块、评价模块等都可以在单体架构中形成清晰边界，便于后续完成模块图、接口说明和 ADR 编写。

5. 保留后续演进空间

虽然当前不采用微服务，但模块化单体可以为未来拆分服务打基础。例如后期可以将消息通知模块、搜索模块或推荐模块独立出来。

为什么不选择其他方案？

不选择微服务架构的原因

1. **复杂度过高** 微服务需要处理服务拆分、服务通信、API 网关、认证传递、分布式事务、部署和监控等问题。
2. **与课程项目目标不匹配** 本项目更强调需求分析、架构设计、核心功能实现和软件工程过程，而不是大规模分布式系统实践。
3. **团队经验不足** 如果团队成员没有微服务项目经验，容易在环境配置和服务联调上消耗过多时间。
4. **容易过度设计** 当前业务规模较小，使用微服务可能让架构复杂度超过实际收益。

不选择事件驱动作为主体架构的原因

1. **核心业务以同步 CRUD 为主** 商品发布、商品浏览、订单创建、评价等功能主要是请求-响应式业务，使用传统分层架构更直接。
2. **增加调试和测试难度** 异步事件链路不如同步调用直观，可能影响开发效率。
3. **引入中间件成本较高** 如果使用消息队列，需要额外配置和维护中间件，不利于快速交付。
4. **适合作为局部补充，而不是主体架构** 事件驱动思想可用于通知、日志、推荐等扩展功能，但当前阶段没有必要作为全局架构。

Prompt 策略效果总结

Prompt 策略	AI 回答特点	质量评价
直接询问推荐架构	回答较快，但偏通用	一般
提供详细项目约束	能结合周期、团队和功能做出较合理建议	很好
要求多方案对比	有助于形成架构候选方案对比表	较好
让 AI 扮演架构师	回答更关注权衡、风险和演进路线	很好

总结

本次实验表明，AI 对架构设计具有一定辅助价值，但回答质量高度依赖 Prompt 的具体程度。直接询问时，AI 往往给出较通用的架构建议；当 Prompt 中包含项目周期、团队人数、功能范围、技术经验等约束后，AI 的建议更加可落地。

最终团队没有完全照搬 AI 的回答，而是结合项目实际情况，选择了更符合课程项目目标的“前后端分离 + 分层架构 + MVC 模式的模块化单体架构”。

最终决策体现的核心工程判断是：

> 对于 10 周内完成的校园二手交易平台，架构设计应优先保证可实现性、可维护性和团队协作效率，而不是追求架构复杂度。